

C More Experiment Details

Experiment setup Each image in BONGARD-LOGO is grey-scale with resolution 512×512 . We train each model in Section 3.1 on the training set for 100 epochs. For a fair comparison, we use ResNet-15 (where feature map sizes are 32-64-128-256-512) with output feature dimension 128 as the backbone network in all the above methods. We use the SGD optimizer with momentum 0.9 and learning rate 0.001 with weight decay $5e-4$. For all meta-learning models, we use a batch size eight on 8 GPUs, namely that each training batch contains eight Bongard problems for the loss computation. For CNN-Baseline, we use a batch size 32 on 8 GPUs. For each run of all the considered models on 8 NVIDIA V100 GPUs, it takes less than 12 hours to get the results. The other hyperparameters are kept the same with the default implementations as the respective original work.

Ablation study on BONGARD-LOGO Here we create a variant of BONGARD-LOGO, where we only include 12,000 free-form shape problems. As the properties of *context-dependent perception* and *analogy-making perception* are not presented any more, concept learning on this variant has a closer resemblance to standard few-shot visual recognition problems [4, 33]. Thus, we expect a large improvement in the performances of these methods. The results of model evaluation and human study in this variant of BONGARD-LOGO, are shown in Table 2. We can see that almost all the considered methods achieve better training and test performances. Specifically, the best training accuracy of methods increases from 81.2% to 96.4%, and the best test accuracy (FF) of methods increases from 66.3% to 74.5%. However, there still exists a large gap between the model and human performance on free-form shape problems alone. It implies the property of *few-shot learning with infinite vocabulary* has already been challenging for current methods. Another observation is that WReN-Bongard outperforms most meta-learning methods on this variant of BONGARD-LOGO, demonstrating its potential as a strong baseline for concept learning and reasoning in simpler cases.

Methods	Train Acc	Test Acc (FF)
SNAIL [19]	74.4 ± 2.5	65.2 ± 2.0
ProtoNet [20]	90.5 ± 0.6	68.5 ± 0.7
MetaOptNet [21]	91.5 ± 0.7	66.9 ± 0.5
ANIL [22]	77.8 ± 0.6	63.2 ± 0.7
Meta-Baseline-SC [23]	92.4 ± 0.3	70.8 ± 0.2
Meta-Baseline-MoCo [23]	95.8 ± 0.3	72.5 ± 0.5
WReN-Bongard [17]	96.4 ± 0.8	74.5 ± 4.0
CNN-Baseline	79.9 ± 6.2	49.8 ± 1.0
Human (Expert)	-	92.1 ± 7.0
Human (Amateur)	-	88.0 ± 7.6

Table 2: Model performance versus human performance in a variant of BONGARD-LOGO which only includes 12,000 free-form shape problems. We report the training and test accuracy (%) on the free-form shape test set (FF). Note that for human evaluation, we report the separate results across two groups of human subjects: *Human (Expert)* who well understand and carefully follow the instructions, and *Human (Amateur)* who quickly skim the instructions or do not follow them at all. The chance performance is 50%.

Capability of the CNN backbone on our visual stimuli Since the images in our benchmark are black-and-white drawings with fine lines, one may have a concern about whether the "visual recognition" part requires different capabilities from what the state-of-the-art CNN models designed for natural images. To evaluate the capability of the CNN backbone on our visual stimuli, regardless of the concept learning and reasoning aspects, we train a standard supervised recognition task with a training set of the visual inputs sampled from our benchmark. In particular, we train two separate binary attribute classifiers for *convex* and *have_two_parts*, respectively, using the same ResNet-15 backbone as in the paper. With each dataset of randomly sampled 14K positive and 14K negative samples, the model achieved the near-perfect train/test accuracies 98.7%/97.0% on *convex* and 98.0%/97.1% on *have_two_parts*. These results demonstrate that the CNN backbone is capable of processing the visual stimuli of our BONGARD-LOGO tasks.

Ablation study on model sizes To show how model sizes affect the concept learning performance, we vary the model size by dividing each layer size in the ResNet-15 backbone by a reduction factor

α , where α is a divisor of the number of parameters. Figure 5 illustrates the training and test results of two top-performing models: ProtoNet and Meta-Baseline-SC. Note that the model size decreases as α increases. We see that both training accuracies decrease consistently with smaller model sizes. On the test sets, the generalization performances also generally get worse as model size decreases, but results slightly vary across different models. ProtoNet is more sensitive to model size than Meta-Baseline-SC: (a) All the test accuracies of ProtoNet tend to decrease with smaller model sizes, while (b) test accuracies of Meta-Baseline-SC mostly remain robust to various model sizes (except for the extreme case of $\alpha = 16$).

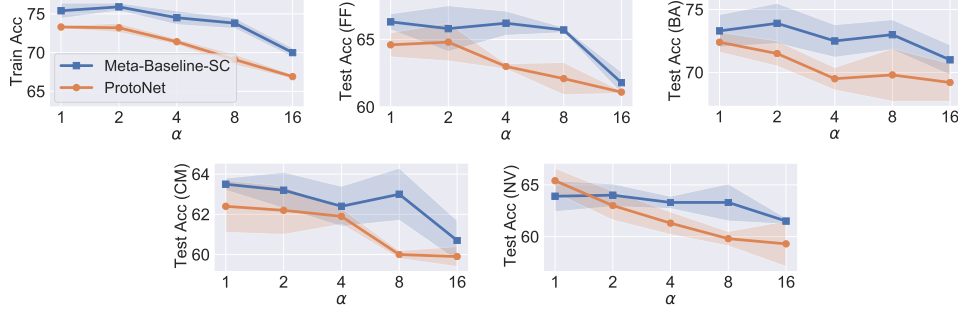


Figure 5: Performance of Meta-Baseline-SC and ProtoNet with different model sizes, controlled by a reduction factor α (i.e., the model size is smaller with a larger α).

Incorporating Symbolic Information for Better Performance Since there is a significant gap between model and human performance in our benchmark, as shown in Table 1, we have discussed the potential of neuro-symbolic approaches to close the gap in Section 5. Here we move one step forward towards a hybrid model and show some preliminary results of incorporating the symbolic information into neural networks. The basic idea is to replace the MoCo pre-training in Meta-Baseline-MoCo with the pre-training of a program synthesis task. We call the new model *Meta-Baseline-PS*, which stands for Meta-Baseline based on program synthesis.

Figure 6 shows the model illustration of (a) Meta-Baseline-PS and (b) one of its components – the *action decoder*. In Meta-Baseline-PS, we first pass the input images into the CNN backbone to extract image features, which are the input of two following branches: 1) the program synthesis module where we use LSTM to convert image features into action features and then use an action decoder to synthesize the action programs; 2) the meta-learner which we use to solve the BONGARD-LOGO problems. The training of Meta-Baseline-PS is composed of two stages, i.e., we first pre-train the program synthesis module to extract the symbolic-aware image feature and then fine-tune it by training the meta-learner.

Methods	Train Acc	Test Acc (FF)	Test Acc (BA)	Test Acc (CM)	Test Acc (NV)
SNAIL [19]	59.2 $\pm$ 1.0	56.3 $\pm$ 3.5	60.2 $\pm$ 3.6	60.1 $\pm$ 3.1	61.3 $\pm$ 0.8
ProtoNet [20]	73.3 $\pm$ 0.2	64.6 $\pm$ 0.9	72.4 $\pm$ 0.8	62.4 $\pm$ 1.3	65.4 $\pm$ 1.2
MetaOptNet [21]	75.9 $\pm$ 0.4	60.3 $\pm$ 0.6	71.7 $\pm$ 2.5	61.7 $\pm$ 1.1	63.3 $\pm$ 1.9
ANIL [22]	69.7 $\pm$ 0.9	56.6 $\pm$ 1.0	59.0 $\pm$ 2.0	59.6 $\pm$ 1.3	61.0 $\pm$ 1.5
Meta-Baseline-SC [23]	75.4 $\pm$ 1.0	66.3 $\pm$ 0.6	73.3 $\pm$ 1.3	63.5 $\pm$ 0.3	63.9 $\pm$ 0.8
Meta-Baseline-MoCo [23]	81.2 $\pm$ 0.1	65.9 $\pm$ 1.4	72.2 $\pm$ 0.8	63.9 $\pm$ 0.8	64.7 $\pm$ 0.3
WReN-Bongard [17]	78.7 $\pm$ 0.7	50.1 $\pm$ 0.1	50.9 $\pm$ 0.5	53.8 $\pm$ 1.0	54.3 $\pm$ 0.6
CNN-Baseline	61.4 $\pm$ 0.8	51.9 $\pm$ 0.5	56.6 $\pm$ 2.9	53.6 $\pm$ 2.0	57.6 $\pm$ 0.7
Meta-Baseline-PS	85.2 $\pm$ 1.0	68.2 $\pm$ 0.3	75.7 $\pm$ 1.5	67.4 $\pm$ 0.3	71.5 $\pm$ 0.5
Human (Expert)	-	92.1 $\pm$ 7.0	99.3 $\pm$ 1.9	90.7 $\pm$ 6.1	
Human (Amateur)	-	88.0 $\pm$ 7.6	90.0 $\pm$ 11.7	71.0 $\pm$ 9.6	

Table 3: Model performance versus human performance in BONGARD-LOGO. We report the test accuracy (%) on different dataset splits, including free-form shape test set (FF), basic shape test set (BA), combinatorial abstract shape test set (CM), and novel abstract shape test set (NV). Note that for human evaluation, we report the separate results across two groups of human subjects: *Human (Expert)* who well understand and carefully follow the instructions, and *Human (Amateur)* who quickly skim the instructions or do not follow them at all. Note that Meta-Baseline-PS means the Meta-Baseline based on program synthesis, which incorporates symbolic information to solve the task. The chance performance is 50%.

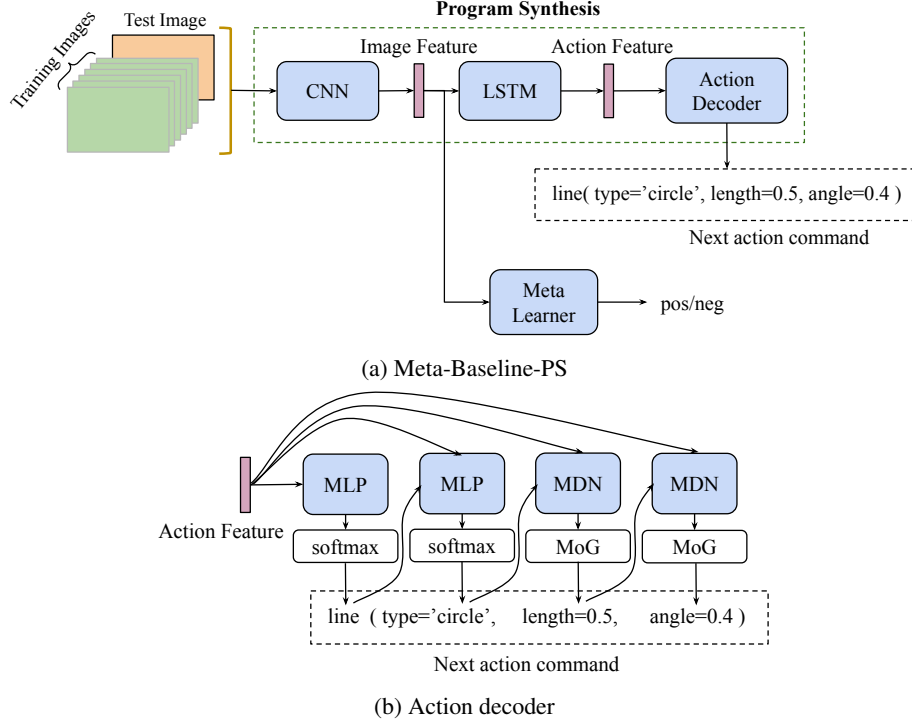


Figure 6: (a) Meta-Baseline-PS, where we first pass the input images into the CNN backbone to extract image features, which are the input of two following branches: 1) the program synthesis module where we use LSTM to convert image features into action features and then use an action decoder to synthesize the action programs; 2) the meta-learner which we use to solve the BONGARD-LOGO problems. (b) the *action decoder*, the architecture for inferring action command from the action feature, where ‘MLP’, ‘MDN’ and ‘MoG’ stand for Multi-Layer Perceptron, Mixture Density Network and Mixture of Gaussians, respectively.

In the action decoder, the action feature from LSTM is passed into each module to sequentially predict each token in the action command, as introduced in Section 2.3. Because the values of *action name* and *moving type* are discrete, we simply use MLP and softmax to predict their values. In contrary, the values of *moving length* and *moving angle* are continuous. To learn prediction with several distinct future possibilities [52], we apply the Mixture Density Network (MDN) [53] together with the Mixture of Gaussians (MoG) parameterizations to predict their values. Experiments have shown MDN achieves better performance than the vanilla L2-norm informed prediction.

Table 3 shows the training and test of Meta-Baseline-PS on BONGARD-LOGO, along with the results of SOTA baseline approaches and human subjects. We can see that Meta-Baseline-PS largely outperforms all the SOTA baselines. It demonstrates that incorporating symbolic information into neural networks improves the overall performance, confirming the great potential of neuro-symbolic methods on tackling the BONGARD-LOGO benchmark. However, by realizing that there is still a large gap between Meta-Baseline-PS and human performance, we leave the exploration of more advanced neuro-symbolic approaches to tackle the challenge of our benchmark, as the future work.

D More Examples in BONGARD-LOGO

We provide more examples from three types of problems, respectively, where each concept in any problem could be about one shape or a combination of two shapes.